Relatório de Evolução de Front-End

Modelo para preenchimento — Laboratório de Informática

Instruções ao aluno

1. Preencha todos os campos com informações do estado atual do seu projeto de front-end.
2. Na Seção 6, detalhe cada ponto de integração com o back-end (endpoints, métodos HTTP, formatos de requisição/resposta e autenticação).
3. Seja objetivo e técnico: utilize os dados reais do projeto (commits, rotas, nomes de variáveis).
4. Anexe capturas de tela quando solicitado e entregue este documento em PDF ou DOCX, conforme orientação do professor.

1. Identificação

Nome do(a) aluno(a)	Deyvid Kauã Viana Alves (04187749), André David (04182774), Pedro Henrique (04145413), Luiz Fernando (04190130), Daniel das Neves (04198142)
Matrícula	04187749 / 04182774 / 04145413 / 04190130 / 04198142
Turma	Engenharia de Software / Ciência da Computação — Laboratório de Práticas Integradas
Disciplina	Laboratório de Desenvolvimento Web e Front-End / Tópicos Especiais
Professor(a)	(A preencher pelo docente)
Data de entrega	20/10/2026
Período / Sprint de referência	Sprint 4 — Ciclo de Integração Front-End & Back-End (Outubro/2026)
Nome do projeto	Maré Alerta — Sistema Preventivo de Alagamentos e Riscos Operacionais para o Comércio de Belém
Link do repositório	Front-End: https://github.com/MonkeyCodeUnama/mare-alerta-frontend Back-End: https://github.com/MonkeyCodeUnama/mare-alerta-backend
Versão / tag / commit de referência	Front-End: v0.2.0-rc (Branch main, ref mare-alerta-frontend-main) Back-End: v0.4.0 / Commit edfe7ad (Branch daniel/backend-oficial)

2. Objetivo do Relatório

Este relatório tem por finalidade registrar e consolidar as informações do estado atual do projeto de front-end do Maré Alerta, desenvolvido no âmbito do Laboratório de Informática. O documento formaliza a arquitetura da aplicação cliente (SPA em React 19 com Vite), a implementação das 7 telas e rotas operacionais (login, cadastro

em duas etapas, dashboard com índice hidrológico, histórico de alertas, checklist preventivo, registro de ocorrências pós-alagamento e configuração do estabelecimento), bem como o detalhamento exaustivo dos pontos de integração com o back-end Spring Boot (contratos de API RESTful, payloads JSON com suporte a @JsonAlias, autenticação stateless JWT com Refresh Token e Rate Limiting). as informações referentes ao estado atual do projeto de front-end desenvolvido em laboratório, documentando o progresso das funcionalidades e, de forma detalhada, os pontos de integração com o back-end (APIs, endpoints, autenticação e tratamento de dados), servindo como instrumento de acompanhamento e avaliação.

3. Estado Atual do Projeto

3.1 Situação geral

Marque a opção que melhor descreve o estado atual do projeto:

( ) Não iniciado ( ) Em desenvolvimento (X) Em testes ( ) Concluído

Percentual estimado de conclusão: 85 % (Telas, navegação, formulários, componentes de interface e contratos de DTOs 100% implementados; fase atual focada no acoplamento das requisições de rede com a API REST).

Última atualização do repositório (data e commit): 12/10/2026 — Commit edfe7ad (Back-End) / 12/09/2026 — Versão mare-alerta-frontend-main.

3.2 Resumo do progresso no período

Descreva, em poucas linhas, o que foi desenvolvido, alterado ou corrigido desde o último relatório:

No período da Sprint 4, a equipe concluiu a refatoração completa do front-end com base no design system voltado ao comércio de Belém/PA:
1. Design System & Componentes: Criação do layout mestre AppShell.jsx com barra lateral de navegação rápida, telemetria resumida da Baía do Guajará (preamar/baixamar), atalhos diretos para Defesa Civil (199) e identidade visual em SVG.
2. Modularização de Telas: Implementação de Home.jsx (Login), Register.jsx (Stepper de 2 etapas), Dashboard.jsx (Painel com gauge de risco e gráfico 12h) e Operations.jsx (Checklist, Histórico, Ocorrências e Estabelecimento).
3. Alinhamento de Contratos de API: Especificação formal em docs/integracao-backend.md alinhando DTOs com @JsonAlias para suporte às propriedades do formulário React.
4. Segurança e Resiliência: Autenticação stateless JWT (Access Token de 2h30 + Refresh Token de 7 dias via POST /auth/refresh), flexibilização cadastral na migração V22 (documento opcional) e tratamento de Rate Limiting por IP (Bucket4j).

3.3 Histórico de alterações relevantes

Data	Alteração realizada	Branch / Commit
12/10/2026	Alinhamento do payload do gráfico hidrológico com o endpoint /telemetria/grafico e telemetria de 10 parâmetros atmosféricos da Open-Meteo.	daniel/backend-oficial / 267e378
12/10/2026	Flexibilização cadastral na migração V22 tornando o documento fiscal opcional no back-end para comerciantes informais.	daniel/backend-oficial / b82d3f1
10/10/2026	Implementação e teste dos contratos de checklist	daniel/backend-oficial /

Data	Alteração realizada	Branch / Commit
	preventivo (/checklists, /checklists/registrar) e hardening de CORS para origens 5173/3000.	316e10a
08/10/2026	Integração do gateway de mensagens WhatsApp (Evolution API v2) no back-end e criação de gatilhos de broadcast no front-end.	akira/developer/DB / 0810e20
11/09/2026	Modularização dos componentes de operações (Checklist, History, Occurrences, Establishment) e implementação do AppShell.	main / 7a3f81e
03/09/2026	Estruturação inicial do Vite, React Router DOM v7, Tailwind CSS v4 e páginas de Login (Home.jsx) e Cadastro em 2 Etapas (Register.jsx).	main / 1f9024a

4. Funcionalidades do Front-End

Liste as funcionalidades previstas e o estado atual de cada uma. Status sugeridos: Não iniciado, Em desenvolvimento, Em testes, Concluído.

Funcionalidade	Descrição	Status	Observações
Autenticação e Acesso (RF-Login)	Formulário de acesso comercial com validação de formato de e-mail, alternador de visibilidade de senha (reveal/hide) e recuperação.	Concluído na UI / Em testes de API	Conectando ao endpoint POST /auth/login para armazenamento de JWT e Refresh Token.
Onboarding em 2 Etapas (RF01–RF04)	Cadastro do comerciante em Stepper: Etapa 1 (Dados do Comércio, CNPJ com máscara e Bairro) e Etapa 2 (Acesso e Senha forte).	Concluído	Validação local ativa; payload compatível com RegisterRequest e @JsonAlias.
Painel de Risco Operacional (RF22)	Dashboard central exibindo índice hidrológico atual, medidor (gauge SVG) de risco moderado/alto, janela de ação segura e insights da Defesa Civil.	Concluído na UI / Em testes de API	Consome dados do endpoint preditivo GET /risco/atual e telemetria consolidada.
Gráfico Fluviométrico 12h (RF07/RF08)	Curva de nível da maré e precipitação pluviométrica (chuva mm/h) com cota de alerta (3.5m) e horário de pico da preamar.	Concluído	Mapeado diretamente para o DTO TelemetriaPontoGraficoDTO (GET /telemetria/grafico).
Checklist de Prevenção	Lista de ações mitigatórias dividida em	Concluído	Mapeado para

Funcionalidade	Descrição	Status	Observações
(RF12–RF14)	fases (Antes da Cheia, Durante a Maré, Após Baixar) com checkboxes de status e responsáveis.		ItemChecklistRequest / ItemChecklistResponse (/checklists).
Histórico e Alertas da Rede (RF11/RF23)	Visualização de comunicados fluviométricos oficiais, filtros por categoria (Maré, Chuva, Comunidade) e botão de emissão emergencial.	Concluído	Alinhado com GET /alertas e despacho via WhatsApp pela Evolution API.
Registro de Ocorrências (RF19–RF20)	Formulário estruturado de registro de sinistro: lâmina d'água (cm), bens atingidos, prejuízo em R$, horas fechado e anexo de fotos.	Concluído	Gera protocolo único (OC-{estab}-{seq}) e prepara upload via multipart/form-data.
Perfil do Estabelecimento (RF01–RF06)	Configuração de dados físicos, localização no mapa da Doca/Belém, grade de horários de funcionamento e seleção de bens críticos.	Concluído	Integração via GET /estabelecimento/me e PUT /estabelecimento/{id}.

5. Telas, Rotas e Componentes

Descreva as telas/rotas implementadas e seus principais componentes.

Tela / Rota	Componentes principais	Estado	Pendências
Login /	Home.jsx, brand-intro (marca d'água e logo), login-card, formulário controlado de e-mail/senha, alternador de senha com SVG, rodapé institucional.	Concluído	Conectar o evento onSubmit com requisição HTTP POST /auth/login.
Cadastro de Comerciante /cadastro	Register.jsx, BrandHeader, Stepper (indicador Etapa 1 / 2), InputField, select de bairros de Belém, business-summary e validação de senha.	Concluído	Conectar o formulário final da Etapa 2 ao endpoint POST /auth/registro persistindo o token JWT retornado.
Dashboard de Risco /dashboard	Dashboard.jsx, AppShell.jsx, salutation com status de telemetria, risk-card (gauge hidrológico), action-card (checklist rápido), chart-card e metric-row.	Concluído	Trocar o mock sintético src/data/tide.js pelo hook de consumo de GET /telemetria/dashboard e GET /risco/atual.
Checklist Operacional	Operations.jsx (Checklist), TaskGroup, barra de progresso de prontidão	Concluído	Conectar a persistência dos checkboxes ao endpoint

Tela / Rota	Componentes principais	Estado	Pendências
/checklist	(readiness), abas temporais de maré e botões de ação coletiva via WhatsApp.		PUT /checklists/{id}.
Histórico de Boletins /historico	Operations.jsx (History), cards de estatísticas (history-stats), barra de filtros por categoria, lista de comunicados (alert-list) e card comunitário.	Concluído	Vincular a listagem dinâmica ao endpoint público GET /alertas.
Declaração de Perdas /ocorrencias	Operations.jsx (Occurrences), seletores de cota d'água, botões múltiplos de categorias de dano, inputs financeiros/temporais e preview de fotos anexadas.	Concluído	Integrar o envio do formulário em POST /ocorrencias e upload de fotos no endpoint POST /ocorrencias/{id}/fotos.
Meu Estabelecimento /estabelecimento	Operations.jsx (Establishment), seletor de tipo comercial, mapa hidrológico de bacia, seletores de dias da semana e tags de itens críticos.	Concluído	Preencher os campos com o retorno de GET /estabelecimento/me e salvar alterações via PUT /estabelecimento/{id}.
Moldura Global (Layout) AppShell	AppShell.jsx, barra lateral (ops-sidebar), indicador de cota da Baía do Guajará, menu de navegação, cabeçalho de alerta e atalho Defesa Civil 199.	Concluído	Injetar dados reais do usuário autenticado no avatar e nome da loja.

6. Pontos de Integração com o Back-End

Esta é a seção central do relatório. Detalhe todas as formas como o front-end se comunica com o back-end, preenchendo cada subseção com os dados reais do projeto.

6.1 Configuração geral da API

URL base — ambiente de desenvolvimento	http://localhost:8080 (Acesso direto) / http://127.0.0.1:8080
URL base — ambiente de produção	https://api.marealerta.belem.br (Orquestração Docker: http://marealerta_app:8080)
Formato de dados (ex.: JSON)	application/json; charset=UTF-8 (e multipart/form-data para fotos)
Tipo de autenticação (ex.: JWT, sessão, API key)	JWT Stateless (Bearer <token>) com claim estrito type: ACCESS + Refresh Token

Biblioteca de requisições (ex.: fetch, axios)	axios (v1.7+) com interceptores centralizados de request/response e renovação de token
CORS configurado? (sim/não/parcialmente)	Sim. Classe CorsConfig.java autoriza localhost:3000, localhost:5173, allowCredentials: true, métodos GET, POST, PUT, DELETE, OPTIONS, PATCH e headers padrão com maxAge 3600s.

6.2 Endpoints consumidos

Preencha uma linha para cada endpoint já consumido (ou planejado) pelo front-end. Status sugeridos: Integrado e testado, Integrado sem teste, Em desenvolvimento, Planejado.

Método	Endpoint	Finalidade	Dados enviados	Resposta esperada	Status
POST	/auth/registro	Criação de conta do comerciante e estabelecimento inicial.	{"email":"...", "senha":"...", "estabelecimento":{...}}	{"token":"...", "refreshToken": "...", ...}	Integrado sem teste
POST	/auth/login	Autenticação do usuário na plataforma.	{"email":"...", "senha":"..."}	{"token":"ey...", "refreshToken": "ey...", ...}	Integrado e testado
POST	/auth/refresh	Renovação automática do Access Token expirado.	{"refreshToken" :"ey..."}	{"token":"eyNo vo...", "refreshToken": "eyNovo...", ...}	Integrado sem teste
GET	/estabelecimento/me	Carrega perfil do comércio do usuário autenticado.	Nenhum (Header Authorization: Bearer)	{"id":1, "nome":"...", "bairro":"...", ... }	Integrado e testado
PUT	/estabelecimento/{id}	Atualização de endereço, horários e itens vulneráveis.	{"nome":"...", "documento":". ..", "bairro":"...", ...}	Objeto EstabeleciментoResponse atualizado.	Integrado e testado
GET	/telemetria/dashboard	Consolidação climática e maré em tempo real.	Nenhum	{"estacaoCodig o":"BELEM_CEN TRO", "nivelAtual":1.8 0, ...}	Integrado e testado
GET	/telemetria/grafico	Série temporal de 24h para gráfico de chuva e maré.	Nenhum	[{"time":"12:00 ", "level":1.20, "rain":0.0, ...}]	Integrado e testado
GET	/risco/atual	Consulta de	Query param: ?	{"bairro":"BELE	Integrado e

Método	Endpoint	Finalidade	Dados enviados	Resposta esperada	Status
		inferência do modelo de Machine Learning.	localidade=BELEM_CENTRO	M_CENTRO", "nivelRisco":"MODERADO", ...}	testado
GET	/alertas	Listagem pública de comunicados e avisos oficiais.	Query param opcional: ?estabelecimentoId=1	[{"id":1, "nivelRisco":"ALTO", "mensagem":"...", ...}]	Integrado e testado
POST	/alertas/bairro	Emissão de alerta emergencial com broadcast no WhatsApp.	{"bairro":"CAMPINA", "nivelRisco":"ALTO", ...}	[{"id":10, "nivelRisco":"ALTO", ...}]	Integrado sem teste
GET	/checklists	Consulta de tarefas de contingência do comércio.	Query params: ?estabelecimentoId=1&statusChecklist=PENDENTE	[{"id":1, "titulo":"...", "status":"PENDENTE", ...}]	Integrado e testado
PUT	/checklists/{id}	Atualização de status da tarefa (concluída/pendente).	{"estabelecimentoId":1, "status":"CONCLUIDO", ...}	Objeto ItemChecklistResponse atualizado.	Integrado e testado
POST	/ocorrencias	Registro estruturado de prejuízos pós-enchente.	{"estabelecimentoId":1, "titulo":"...", ...}	{"id":1, "protocolo":"OC-1-1", "status":"RASCUNHO", ...}	Integrado sem teste
POST	/ocorrencias/{id}/fotos	Envio de fotos comprobatórias do sinistro.	multipart/form-data com parâmetro 'arquivo'	{"id":1, "nomeArquivo":"...", "urlFoto":"...", ...}	Integrado sem teste

Exemplo de preenchimento: POST | /api/login | Autenticação do usuário | { email, senha } | { token, usuario } | Integrado e testado

6.3 Autenticação e autorização

Onde o token/credencial é armazenado no front-end (ex.: localStorage, cookie, memória): localStorage do navegador sob as chaves marealerta_access_token, marealerta_refresh_token e marealerta_user (com migração planejada para cookies HttpOnly em produção).

Como o token é enviado nas requisições (ex.: header Authorization: Bearer): Cabeçalho HTTP padrão de autorização Authorization: Bearer <token>, injetado automaticamente via interceptor global do Axios.

Há renovação de token (refresh token)? Descreva o fluxo: Sim. O back-end emite par de tokens (Access Token com validade de 2h30 e Refresh Token de 7 dias). Ao interceptar erro HTTP 401, o front-end envia POST /auth/refresh com o refresh token, armazena o novo par retornado e repete a requisição original de forma transparente.

Quais rotas/telas são protegidas e como o controle de acesso é feito: As rotas /dashboard, /checklist, /historico, /ocorrencias e /estabelecimento são protegidas pelo componente <ProtectedRoute />. Sem token válido, o usuário é redirecionado para a raiz (/); as rotas / e /cadastro são públicas.

6.4 Tratamento de erros e estados de carregamento

Descreva como o front-end reage a falhas de comunicação com o back-end (erros 4xx/5xx, timeout, rede indisponível) e como os estados de carregamento (loading) são exibidos ao usuário:

1. Erros de Validação (HTTP 400): A API devolve o formato padronizado ApiError; as mensagens são capturadas e apresentadas visualmente sob os respectivos campos nos formulários.
2. Falhas de Autenticação (HTTP 401): Disparo automático da renovação silenciosa com refresh token; caso expire, redireciona o usuário para a tela de login.
3. Conflitos Cadastrais (HTTP 409): Apresentação de alerta explicativo de e-mail ou documento já existente na base de dados.
4. Limitação de Taxa (HTTP 429 - Rate Limit Bucket4j): O front-end captura o código 429, extrai o tempo do cabeçalho Retry-After e exibe um modal bloqueador instruindo o usuário a aguardar os segundos indicados.
5. Falhas de Rede e Servidor (HTTP 5xx / Timeout / Offline): Exibição de banner de aviso no topo da interface, ativando cache local dos últimos dados telemétricos recolhidos para manter a prontidão operacional durante intempéries.
6. Estados de Carregamento (Loading): Utilização de skeletons animados nos cards e no gráfico do Dashboard, e desativação dos botões com spinner (ex.: 'Carregando...', 'Salvando...') para prevenir múltiplos envios concorrentes.

6.5 Variáveis de ambiente e configuração

Variável (ex.: VITE_API_URL)	Finalidade	Ambiente
VITE_API_URL	URL base para consumo de todos os endpoints RESTful do back-end.	Dev: http://localhost:8080 | Prod: https://api.marealerta.belem.br
VITE_WS_WHATSAPP_URL	Endpoint da Evolution API para monitoramento do QR Code da instância.	Dev: http://localhost:8081 | Prod: https://whatsapp.marealerta.belem.br
VITE_APP_NAME	Nome do sistema exibido nas tags de título e cabeçalho da aplicação.	Todos: Maré Alerta Belém
VITE_TIMEOUT_MS	Limite de tempo de espera (timeout) para	Todos: 8000 (8

Variável (ex.: VITE_API_URL)	Finalidade	Ambiente
	requisições antes do fallback.	segundos)

7. Dificuldades Encontradas e Soluções

Problema / Dificuldade	Causa identificada	Solução aplicada (ou prevista)	Status
Divergência de nomenclatura nos campos (Front em Inglês vs Back em Português)	O protótipo React utilizava chaves como name, cnpj, neighborhood, street, opening, closing, enquanto o Spring Boot adotou português (nome, documento, bairro, logradouro).	Configuração de anotações @JsonAlias do Jackson nos DTOs do back-end, aceitando ambos os formatos sem necessidade de refatorar a interface.	Solucionado
Discrepância de Timezone e Hora Cheia na Inferência de IA	O microsserviço preditivo (mare-alerta-ia / FastAPI) rejeitava com HTTP 422 requisições sem timestamp truncado na hora cheia, além do conflito GMT-3 (Belém) vs UTC (Docker).	Padronização em java.time.Instant (TIMESTAMPTZ) no back-end e conversão explícita no front-end para o fuso horário America/Belem.	Solucionado
Bloqueio por Rate Limiting durante testes intensivos	O filtro RateLimitFilter (Bucket4j) bloqueava o front-end emitindo HTTP 429 devido a chamadas duplicadas originadas pelo React StrictMode em desenvolvimento.	Ajuste da quota local para 60 requisições/minuto e implementação de cache com stale-time de 30 segundos no front-end para dados de telemetria.	Solucionado
Comerciantes informais sem CNPJ	A modelagem inicial exigia CNPJ obrigatório, inviabilizando o registro de pequenos feirantes e barracas do Ver-o-Peso.	Criação da migração Flyway V22 tornando o documento opcional no back-end e ajuste no front-end para aceitar CPF ou avançar sem preenchimento fiscal.	Solucionado

8. Próximos Passos

Tarefa	Prioridade	Prazo previsto	Depende do back-end?
Instalação e configuração do cliente Axios com interceptor de Refresh Token	Alta	24/10/2026	Não
Substituição das fixtures de maré (src/data/tide.js) pelos endpoints reais de telemetria	Alta	27/10/2026	Sim (Endpoints já disponíveis)
Implementação do fluxo de upload real de fotos de sinistros (multipart/form-data)	Média	31/10/2026	Sim (Endpoint já disponível)
Configuração de Service Worker e manifesto PWA para alertas e operação offline	Média	05/11/2026	Não
Conexão dos botões de disparo de alerta ao gateway de WhatsApp (Evolution API)	Alta	08/11/2026	Sim (Instância já orquestrada)

9. Anexos e Evidências

Liste aqui os anexos entregues junto ao relatório (capturas de tela das telas, exemplos de requisições/respostas, links de deploy ou gravação de demonstração):

- Especificação de Requisitos Funcionais e Telas: Documentos de referência requisitos_mare_alerta.md e paginas_mvp_mare_alerta.md.
- Contratos e Arquitetura de Integração: Documentos docs/integracao-backend.md e docs/GUIA_BACKEND.md.
- Documentação Interativa da API (Swagger UI): Disponível localmente em http://localhost:8080/swagger-ui/index.html e esquema OpenAPI em http://localhost:8080/v3/api-docs.
- Design Tokens e Artefatos Visuais: Pasta src/assets/figma/ contendo os vetores SVG do medidor de maré (gauge), ícones de alerta e o mapa hidrológico da calha da Doca (map.png).

10. Uso do Professor — Avaliação

Critério	Pontuação máxima	Nota atribuída
Atualização correta do estado do projeto	2,0	
Detalhamento dos pontos de integração com o back-end	4,0	
Clareza e organização do documento	2,0	
Evidências e anexos	2,0	

Critério	Pontuação máxima	Nota atribuída
Total	**10,0**	

Comentários do professor:

_____
_____

_____
_____

_____
Assinatura do(a) aluno(a)

_____
Assinatura / carimbo do(a) professor(a)